DIGITAL SIGNATURE SYSTEM

Implementation Using Python

Public-Key Cryptography & RSA Digital Signatures

Subject	Algorithm	Language	Key Size
Cryptography & Network Security	RSA + SHA-256	Python 3.x	2048-bit RSA

1. Objective

This activity implements a complete Digital Signature System using Python to demonstrate the principles of public-key cryptography. Students will learn how to:

- Generate an RSA key pair (public key and private key)
- Sign a message using the private key combined with SHA-256 hashing
- Verify the digital signature using the public key
- Understand message integrity and authentication in cryptographic systems

2. Background Theory

2.1 Public-Key Cryptography

Public-key cryptography (asymmetric cryptography) uses a mathematically linked key pair: a public key (shared openly) and a private key (kept secret). Data encrypted with one key can only be decrypted with the other.

2.2 RSA Algorithm

RSA (Rivest–Shamir–Adleman) is the most widely used asymmetric cryptosystem. It is based on the mathematical difficulty of factoring the product of two large prime numbers. Key properties:

- Key Generation: Two large primes p and q → n = p×q; public exponent e and private exponent d
- Signing: signature = m^d mod n (using private key)
- Verification: m = signature^e mod n (using public key)

2.3 SHA-256 Hashing

SHA-256 (Secure Hash Algorithm 256-bit) produces a fixed 256-bit digest of any input. Properties used in digital signatures:

- Deterministic — same input always gives same output
- One-way — cannot reverse the hash to get original data
- Avalanche effect — tiny input change = completely different hash
- Collision resistant — computationally infeasible to find two inputs with same hash

2.4 Digital Signature Flow

SIGNING (Sender)	VERIFICATION (Receiver)

1. Original Message 2. Hash with SHA-256 → digest 3. Encrypt digest with Private Key 4. Attach signature to message	1. Receive message + signature 2. Decrypt signature with Public Key → digest1 3. Hash message with SHA-256 → digest2 **4. If digest1 == digest2 → VALID ✓**

3. Requirements & Setup

3.1 Software Requirements

- Python 3.7 or higher
- cryptography library (pip install cryptography)

3.2 Installation

```
# Install required library
pip install cryptography

# Verify installation
python -c "from cryptography.hazmat.primitives.asymmetric import rsa; print('OK')"
```

4. Python Source Code

The complete implementation is organized into four main functions: key generation, signing, verification, and demonstration.

4.1 digital_signature.py — Complete Source

```
# ============================================================
#  Digital Signature System — RSA + SHA-256
#  Demonstrates: Key Generation, Signing, Verification
# ============================================================

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.exceptions import InvalidSignature
import base64
```

```python
# ── STEP 1: Key Generation ──────────────────────────────────
def generate_key_pair():
    """Generate a 2048-bit RSA key pair."""
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )
    public_key = private_key.public_key()
    return private_key, public_key


# ── STEP 2: Sign Message ────────────────────────────────────
def sign_message(private_key, message: str) -> bytes:
    """
    Sign a message using the private key.
    Internally hashes the message with SHA-256 (PSS padding).
    """
    message_bytes = message.encode('utf-8')
    signature = private_key.sign(
        message_bytes,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    return signature


# ── STEP 3: Verify Signature ────────────────────────────────
def verify_signature(public_key, message: str, signature: bytes) -> bool:
    """
    Verify the signature using the public key.
    Returns True if valid, False if tampered.
    """
    try:
        message_bytes = message.encode('utf-8')
        public_key.verify(
            signature,
            message_bytes,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return True
    except InvalidSignature:
        return False
```

```
# ── STEP 4: Export Keys (PEM format) ──────────────────────────
def export_keys(private_key, public_key):
    """Serialize keys to PEM strings for display/storage."""
    priv_pem = private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    ).decode('utf-8')

    pub_pem = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ).decode('utf-8')

    return priv_pem, pub_pem


# ── MAIN: Demonstration ────────────────────────────────────────
if __name__ == '__main__':
    print('=' * 60)
    print('   DIGITAL SIGNATURE SYSTEM — RSA + SHA-256')
    print('=' * 60)

    # Step 1: Generate key pair
    print('\n[1] Generating 2048-bit RSA Key Pair...')
    private_key, public_key = generate_key_pair()
    priv_pem, pub_pem = export_keys(private_key, public_key)
    print('    Private Key (first 64 chars):', priv_pem[27:91] + '...')
    print('    Public  Key (first 64 chars):', pub_pem[26:90] + '...')

    # Step 2: Sign a message
    message = 'Hello, this message is signed with RSA and SHA-256!'
    print(f'\n[2] Original Message: "{message}"')
    signature = sign_message(private_key, message)
    sig_b64 = base64.b64encode(signature).decode('utf-8')
    print(f'    Signature (Base64): {sig_b64[:64]}...')

    # Step 3: Verify — original message
    print('\n[3] Verification Tests:')
    result = verify_signature(public_key, message, signature)
    print(f'    Original message  → Signature Valid: {result}')

    # Step 4: Tampered message test
    tampered = message + ' (tampered)'
    result_tampered = verify_signature(public_key, tampered, signature)
    print(f'    Tampered message  → Signature Valid: {result_tampered}')

    print('\n' + '=' * 60)
    print('   DEMONSTRATION COMPLETE')
    print('=' * 60)
```

5. Code Explanation

5.1 Key Generation

The generate_key_pair() function creates a 2048-bit RSA private key using a public exponent of 65537 (standard value). The public key is derived from the private key automatically.

Why 65537?	65537 (= 2^16 + 1) is a Fermat prime — it's computationally efficient to use in RSA and offers good security properties. It is the standard recommended public exponent.

5.2 Message Signing

The sign_message() function uses RSA-PSS (Probabilistic Signature Scheme) padding with SHA-256:

- The message is first encoded to bytes (UTF-8)
- SHA-256 hashes the message → produces a 256-bit digest
- The private key encrypts the digest using PSS padding
- Result is a 256-byte (2048-bit) signature

5.3 Signature Verification

The verify_signature() function reverses the process using the public key:

- Decrypts the signature using the public key
- Independently computes SHA-256 hash of the received message
- Compares the two digests — if they match, signature is VALID
- InvalidSignature exception is raised if tampered → returns False

5.4 Why PSS Padding?

PSS (Probabilistic Signature Scheme) is preferred over older PKCS#1 v1.5 because it includes a random salt, making it provably secure and resistant to certain attacks. It is the modern recommended RSA signature scheme.

6. Expected Output (Screenshot Simulation)

When the program is executed, the following output is displayed in the terminal:

```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

   DIGITAL SIGNATURE SYSTEM — RSA + SHA-256
============================================================

[1] Generating 2048-bit RSA Key Pair...
    Private Key (first 64 chars):
MIIEvAIBADANBgkqhkiG9w0BAQEFAASCBKYwggSiAgEAAoIBAQCpmNuCKQyiixP...
    Public  Key (first 64 chars):
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAqZjbgikMoosT7GJZjoU...

[2] Original Message: "Hello, this message is signed with RSA and SHA-256!"
    Signature (Base64): S98zqatfR6J2z+ZgdNCOFpPW3Stx0Dv7YEmunsT/9tRjXDlI/5UX99qreyeV74km...

[3] Verification Tests:
    Original message  → Signature Valid: True
    Tampered message  → Signature Valid: False

============================================================
   DEMONSTRATION COMPLETE
============================================================
```

Test Case	Input	Expected Result
Valid Signature	Original message + correct signature	**True ✓ (Valid)**
Tampered Message	Modified message + original signature	**False ✗ (Invalid)**
Wrong Key	Different public key for verification	**False ✗ (Invalid)**

---

7. Security Analysis

7.1 Security Properties Achieved

Property	How It Is Achieved	Mechanism
Authentication	Only the private key holder can create a valid signature	RSA Private Key
Integrity	Any message change invalidates the signature	SHA-256 Hash
Non-repudiation	Signer cannot deny signing — only they had the private key	Asymmetric Keys
Confidentiality	Not provided (separate encryption needed)	N/A

7.2 Attack Resistance

- Brute-force attack: 2048-bit RSA keys provide ~112-bit security level — computationally infeasible
- Hash collision attack: SHA-256 has 2^128 collision resistance — no practical attack known
- Message tampering: Any modification produces a completely different SHA-256 hash
- Replay attack: Timestamps or nonces can be added to signatures to prevent reuse